Quantization Calibration and Speculative Decoding for On-Device Thai LLMs: A Measurement Study

Watin Promfiy
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
66070184@kmitl.ac.th

Atibodee Kuiprasert
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
66070216@kmitl.ac.th

Lapas Pradittasnee
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
lapas@it.kmitl.ac.th

***Abstract*—The local large-language-model toolchain—convert to GGUF, quantize to roughly four bits, serve with `llama.cpp` or `ollama`—was built by an English-speaking community, and its defaults quietly disadvantage Thai. Measuring 1B- and 3B-parameter models on a 16 GB Apple M4, with a replication on an x86-64 CPU, we report four results, one of which contradicts our own starting hypothesis. (i) Thai pays a large, invisible throughput tax: on professionally translated parallel passages it costs 2.20 times as many tokens as English, only 1.09% of the Llama-3.2 vocabulary contains a Thai character, and three released Thai model families ship their base tokenizer unmodified; end to end, `llama.cpp` delivers 400.7 English characters per second against 147.8 Thai. (ii) Calibration language matters only when bits are scarce: the community-standard importance-matrix corpus contains zero Thai characters, yet replacing it with Thai text is statistically indistinguishable from English Wikipedia at Q4_K_M and significantly *worse* at Q3_K_M (paired t-test, t = 15.4); only at Q2_K does matching pay, recovering 5.75% of Thai perplexity at no English cost. (iii) The tax is partly self-refunding: because Thai is cut into more, individually more predictable tokens, model-free speculative decoding speeds Thai up by 7% while slowing English down by 6% with the same model and task on the M4. (iv) We replace the usual qualitative account of when speculation pays with a measured cost curve for wide verification passes and an exact speedup identity, and show on a second backend and a model with no Thai tuning that the language effect replicates (a 44% speedup for Thai against 12% for English) while the cost of verification, the best drafter, and exact losslessness do not transfer.**

***Index Terms*—on-device inference, quantization, tokenization, speculative decoding, Thai natural language processing, large language models**

I. Introduction

On-device inference matters for Thai for the usual reasons—privacy in clinical and legal settings, cost in public-sector deployment, and latency—and the practical toolchain is GGUF weights served by `llama.cpp` [1] or `ollama` [2]. Two of that toolchain's defaults are shaped by English.

The calibration default. Modern `llama.cpp` quantization is importance aware: an *importance matrix* (imatrix), estimated by running the full-precision model over a calibration corpus, records how much each weight channel matters, and the quantizer spends its bit budget accordingly—the same principle established for large language models by GPTQ [3] and AWQ [4]. The calibration corpus is therefore a hyperparameter deciding *which language's behaviour is preserved*. It is not treated as one. A single file, `calibration_datav3.txt`, produces a large share of the GGUF quantizations published on public model hubs; it contains 274,760 characters, of which **zero** are Thai.

The reporting default. Throughput is reported in tokens per second, which overstates delivered content for any language split into many subword tokens. Thai, written without inter-word spaces, is such a language and unusually so.

We set out to show that Thai-language calibration fixes the first problem. It mostly does not, and Section IV reports that negative result together with the bit-width threshold at which it becomes false. The second problem turns out to contain its own partial mitigation (Section V), which Section VI then puts on a quantitative footing and tests on a second backend. Fig. 1 states the mechanism the paper turns on.

All measurements use 1B- and 3B-parameter models that share the Llama-3.2 tokenizer, on one Apple M4 and one x86-64 CPU. Our conclusions are claims about that setting; Section VII sets out which of them we expect to carry beyond it and which remain to be tested.

II. Related Work

Post-training quantization. GPTQ [3] and AWQ [4] established that calibration data shapes quantization quality, AWQ specifically that activation-derived salience beats weight magnitude for deciding what to protect; SmoothQuant [5] and LLM.int8() [6] address the outlier structure that makes low-bit inference hard, and [7] characterises the accuracy–bit-width trade-off. `llama.cpp`'s k-quant and IQ families adopt the importance-weighting principle via the imatrix. This literature calibrates and evaluates overwhelmingly in English; what happens when the deployment language differs from the calibration language is largely unexamined, and that gap is what Section IV measures.

Multilingual tokenization. That subword vocabularies fragment under-represented languages more heavily is established: [8] ties tokenizer quality to downstream monolingual performance, [9] shows the resulting disparity is large and systematic across languages, and [10] turns it into a monetary and latency cost under commercial APIs. Our contribution is

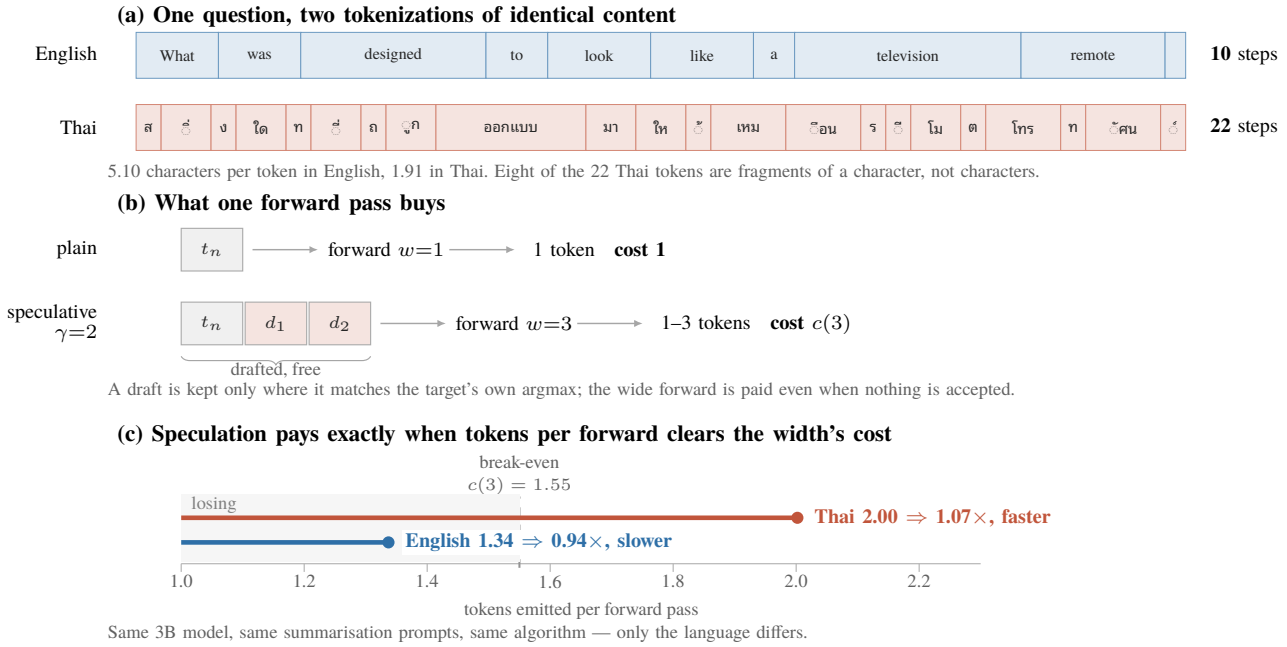


Fig. 1. Why the tokenization penalty partly refunds itself. **(a)** A parallel Belebele question tokenized by the target model's own tokenizer; both bars carry the same content, so the extra cuts are pure overhead. This example's inflation factor of 2.20× is exactly the corpus mean of Table I, so it is representative rather than selected. **(b)** A speculative step verifies the drafted tokens in one wider forward, which costs more than a single-token decode but not proportionally more. **(c)** Profitability therefore turns on whether tokens emitted per forward clear the cost of a speculative step; Thai clears it where English does not, on the same model, prompts and algorithm. Panel (c) plots measured M4 values from Table IV. The dashed line is the single-point estimate $c(3) \approx 1.55$ of Section V; the exact bar is each configuration's mean step cost $\bar{c}$ (Eq. (3), Table V), 1.86 for Thai and 1.43 for English, and each language falls on the same side of its own bar.

not the phenomenon but its consequences for one deployment: we quantify it for Thai on strictly parallel content, tie it to a metric users experience (delivered characters per second), show that current Thai releases do not mitigate it, and show that it interacts with a decoding algorithm in the language's favour.

Speculative decoding. Speculative decoding [11], [12] trades cheap draft proposals against a batched verification pass, losslessly under the standard acceptance rule; blockwise parallel decoding [13] is its antecedent, and Medusa [14], lookahead decoding [15] and SpecInfer [16] vary how drafts are produced and verified. Model-free variants—prompt lookup [17] and n-gram drafting—avoid a draft model entirely. Existing analyses of when speculation pays are framed in terms of the acceptance rate and a draft-to-target cost ratio treated as a constant. Section VI measures that ratio instead, as a function of verification width, and finds it neither constant nor monotonic. To our knowledge the interaction between tokenizer fertility and speculative decoding profitability has not previously been measured.

III. The On-Device Thai Penalty

A. Token inflation

Comparing token counts across languages requires content held constant. We use the 488 unique Belebele [18] passages, which are the FLORES-200 passages [19], professionally translated so that every language expresses identical content, and define the *Token Inflation Factor*

$$\mathrm{TIF}(\ell) = \frac{\overline{|\mathrm{tok}(\ell)|}}{\overline{|\mathrm{tok}(\mathrm{en})|}}, \tag{1}$$

the factor by which a decoder must run more autoregressive steps to deliver the same information. At a fixed tokens-per-second rate, user-perceived speed is divided by TIF.

TABLE I
Token inflation factor (TIF) versus English on parallel content, with vocabulary coverage. "Thai" counts vocabulary entries containing at least one Thai character. Th, Vi, Id, Km, My: Thai, Vietnamese, Indonesian, Khmer, Burmese.

Tokenizer	$\lvert V\rvert$	Thai	%	Th	Vi	Id	Km	My
Llama-3.2 / Typhoon2	128,256	1,394	1.09	**2.20**	1.38	1.54	8.87	11.69
Qwen2.5 / Qwen3	151,665	2,571	1.70	**2.57**	1.42	1.54	6.60	8.98
Gemma-3	262,145	2,177	0.83	**1.60**	1.36	1.14	2.82	2.42

Table I reports the result. Under the Llama and Qwen tokenizers, Thai costs 2.2–2.6× more decode steps than English for identical content: 2.13 Thai characters per token against 4.90. Three points follow.

Thai models inherit the penalty unchanged. Typhoon2-1B and -3B [20] tokenize byte-identically to stock Llama-3.2 [21]; Typhoon2.5 and OpenThaiGPT-1.5 tokenize byte-identically to stock Qwen [22]. Despite substantial Thai continued pretraining, no released model in our set extends the vocabulary.

TABLE II
DELIVERED CONTENT ON IDENTICAL WEIGHTS AND PROMPTS, TYPHOON2-1B Q4_K_M. EQUAL TOKENS PER SECOND IS NOT EQUAL TEXT.

Runtime	Language	TTFT (ms)	tok/s	chars/s
`ollama`	Thai	395.1	66.54	142.3
`llama.cpp`	Thai	379.9	69.14	147.8
`ollama`	English	236.2	78.75	370.3
`llama.cpp`	English	135.1	85.41	**400.7**

The penalty tracks writing system more than resource level. Space-delimited Vietnamese and Indonesian sit at 1.38–1.54, while the *scriptio continua* languages—Thai (2.20), Khmer (8.87) and Burmese (11.69)—are the outliers. This is a statement about token counts only; the experiments of Sections IV–VI are run on Thai alone.

It is a design choice. Gemma-3 [23], with a $2\times$ larger vocabulary, halves Thai's penalty to 1.60.

An independent confirmation fell out of corpus construction: matching Thai and English drafter corpora by *token* count required 8.58 M Thai characters against 17.9 M English, a $2.09\times$ ratio that reproduces TIF on entirely different text.

B. The delivered-content gap

Token counts only matter if they survive to the user. We served identical GGUF weights under `ollama` and `llama-server` with byte-identical prompt strings (the chat template applied once, offline; `ollama` driven with `raw:true`) and greedy sampling everywhere, so the two runtimes must emit identical text. Measured agreement was $28/28$ on Thai and $27/28$ on English.

Table II shows English delivering $2.71\times$ more characters per second than Thai. Two penalties compose: Thai needs more tokens per unit content, *and* it runs fewer tokens per second (69 against 85) because a Thai prompt is $1.8\times$ longer in tokens (204 against 114 on average over the same 28 prompts), enlarging the KV cache and slowing every decode step. Time-to-first-token shows the same effect (380 ms against 135 ms). `ollama` is not a straw-man baseline here: its throughput is within 4% of `llama-server` on Thai and 8% on English.

C. Fragmentation lowers per-token entropy

Thai's fragmentation makes each token individually easier to predict: on held-out Wikipedia, Typhoon2-1B reaches perplexity 4.45 on Thai against 10.17 on English. This does *not* make Thai the easier language. Per-token perplexity is not comparable across languages, since a Thai token carries 2.13 characters where an English token carries 4.90; converting to the language-neutral measure reverses the ordering, at 1.010 bits per character for Thai against 0.683 for English. Thai is not easier—it is cut into easier pieces. Section V exploits exactly that.

IV. DOES CALIBRATION LANGUAGE MATTER?

A. Hypothesis

At fixed bits-per-weight, an importance matrix estimated on English preserves English better than Thai, and substituting Thai calibration recovers Thai quality at no inference cost. We report below that, for a 1B model, this is false from Q3_K_M upward (4.4–5.2 effective bits per weight) and true at Q2_K (3.7).

B. Design

Five calibration conditions separate three otherwise confounded variables (language, corpus size, domain): `none` (round-to-nearest, no imatrix); `default` (`calibration_datav3.txt`, 125 chunks of 512 tokens), the *ecological* baseline—what Thai users download today; `en` (English Wikipedia, 625 chunks), the *controlled* baseline, matched to `th` in size and domain; `th` (Thai Wikipedia, 625 chunks); and `mix` (50/50, 628 chunks). Since `default` differs from the others in both language and size it cannot isolate a language effect; the clean language-only contrast is `en` versus `th`. Calibration and evaluation text are disjoint at the article level. The model is Typhoon2-1B-Instruct, quantized to Q4_K_M, IQ4_XS, Q3_K_M and Q2_K. We name levels by their nominal type (Q2_K is the "2-bit" level); effective bits per weight (Table III) are higher because `llama.cpp` stores the 128k-row embedding matrix, a fifth of this model's weights, in Q6_K at every level.

C. Verifying the manipulation

Quantizer logs confirm the intended imatrix was loaded for each of the 21 cells; the five Q4_K_M variants have distinct SHA-1 hashes, so the weights genuinely differ; and `llama.cpp` reports *identical* bits-per-weight across calibration conditions (5.180 at Q4_K_M, 4.420 at Q3_K_M, 3.710 at Q2_K), so any quality difference is obtained at exactly equal file size and inference cost. Re-running a quantization reproduced a byte-identical file. IQ4_XS is not bpw-matched to its own `none` row (4.790 against 4.760), because the IQ family changes tensor-type selection when no imatrix is supplied; we draw no conclusion from that row.

D. Statistics

Independent confidence intervals badly under-detect differences between two quantizations of the *same* model on the *same* data, because their errors are strongly correlated. We pair everything. Per-chunk log-losses are recovered by inverting the running-perplexity trace that stock `llama.cpp` already prints,

$$\bar{\ell}_n \;=\; n \ln \mathrm{PPL}_n - (n-1) \ln \mathrm{PPL}_{n-1}, \tag{2}$$

and tested pairwise over 260 chunks; per-item correctness is recovered from the running-accuracy trace and tested with an exact-binomial McNemar test. Both traces are free—no extra compute. The inversion reproduces the reported aggregate exactly (recovered mean log-loss $1.4930 \Rightarrow$ perplexity 4.4505, against a reported 4.4505).

E. Results

Table III and Fig. 2 give three readings.

(a) From Q3_K_M upward, language matching does not help and can hurt. At Q4_K_M the `en`/`th` difference is null

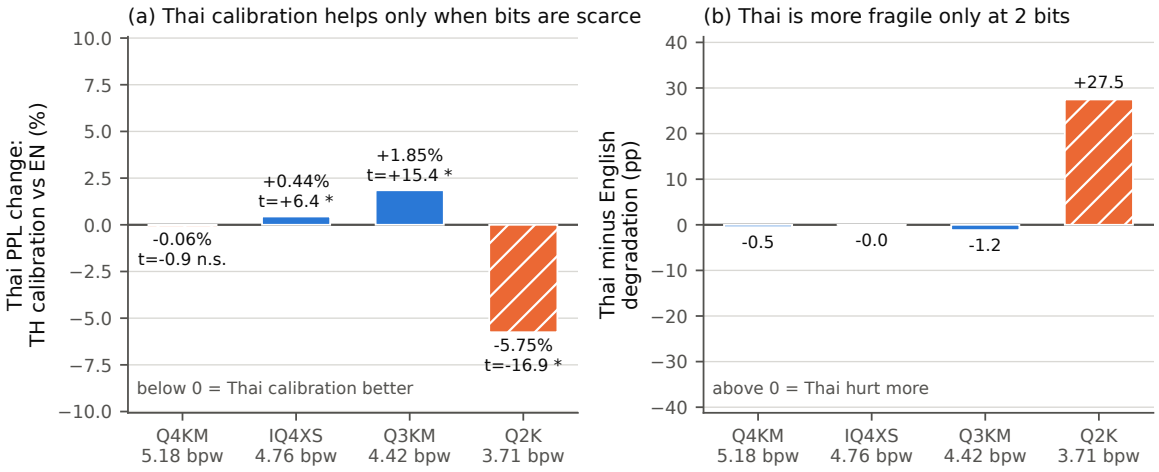


Fig. 2. Both calibration findings are bit-width dependent. **(a)** Paired per-chunk change in Thai perplexity from switching calibration from English to Thai Wikipedia at identical bits-per-weight; bars below zero favour Thai. **(b)** Thai degrades more than English only at Q2_K, so the common claim that quantization penalises low-resource languages is itself bit-width dependent. Panel (b) shows the Thai-calibrated models; without an imatrix the Q2_K gap widens to 80.6 points (Section IV-E).

TABLE III
PAIRED PER-CHUNK TEST OF THAI VERSUS ENGLISH WIKIPEDIA CALIBRATION AT IDENTICAL BITS-PER-WEIGHT (BPW), TYPHOON2-1B. NEGATIVE MEANS THAI CALIBRATION IS BETTER. THE EFFECT REVERSES SIGN AT Q2_K.

		on Thai text		on English text	
Quant	bpw	ΔPPL	t	ΔPPL	t
Q4_K_M	5.180	−0.06%	−0.94	+0.23%	+2.34
IQ4_XS	4.760	+0.44%	+6.41	+0.35%	+4.85
Q3_K_M	4.420	+1.85%	+15.36	+1.91%	+17.55
Q2_K	**3.710**	**−5.75%**	**−16.94**	+0.11%	+0.41

(t=−0.94 over 260 paired chunks). At Q3_K_M and IQ4_XS, English Wikipedia calibration is significantly *better for Thai* than Thai calibration. We did not expect this; Section VII gives the leading confound.

(b) At Q2_K, language matching pays, and pays substantially. Thai calibration improves Thai perplexity by 5.75% over English calibration (t=−16.94) with no measurable English cost (t=+0.41). The reading we favour is that the importance matrix only binds when the bit budget is genuinely scarce: at four bits there is enough precision that where you spend it matters little, and the corpus is effectively a free parameter. Where that point lies is a property of the model as well as the quantizer; we have located it for one 1B model only (Section VII).

(c) Thai fragility is likewise bit-width dependent. Under round-to-nearest, Thai degrades +263.9% against English's +183.3% at Q2_K—an 80.6 percentage-point asymmetry—but at Q4_K_M Thai degrades *less* than English. The blanket claim that quantization hurts low-resource languages more holds only at aggressive bit-widths.

Downstream benchmarks are uninformative at this scale, and we report them as such rather than as null results: FP16 scores 38.9% on Belebele-th against a 25% chance floor and 24.8% on ThaiExam [24] against 21.1%, and no McNemar comparison between calibration conditions reaches significance (smallest p=0.09 across Belebele-th/en, XCOPA-th [25] and ThaiExam). A 1B model lacks the headroom for these tasks to resolve a sub-1% perplexity effect, so the practical weight of this section rests on perplexity alone.

Practical upshot. If you ship a Q2_K Thai model of this size, calibrate in Thai—it is a one-line change, costs nothing at inference, and buys about 5.8% perplexity. At Q4_K_M, the calibration language makes no measurable difference.

V. TOKEN INFLATION IS SELF-MITIGATING

A. Setup

We compare four drafters: `none` (the greedy baseline), `lookup` (prompt lookup [17]: copy whatever followed the most recent earlier occurrence of the last k tokens), `ngram` (a static, confidence-gated token n-gram model with stupid backoff over orders 1–4, trained offline on 4 M tokens of Wikipedia *tokenized by the target model's own tokenizer*), and `hybrid` (`lookup`, falling back to `ngram`). Thai and English each get their own table, built identically from corpora matched by *token* count (4,008,138 against 4,005,592); matching by characters would have handed the Thai drafter roughly half the training signal and made any Thai advantage an artefact of corpus size. Verification accepts a draft token only where it equals the target model's own argmax, so the emitted sequence is the greedy one, up to the floating-point caveat of Section VI-C.

B. Two measurement problems fixed first

Mode ordering was confounded with thermal state. Running each drafter to completion in turn measures the last one on a hotter, slower machine. This was not hypothetical:

TABLE IV
SPECULATIVE DECODING AT γ=2 ON TYPHOON2-3B SUMMARIZATION, APPLE M4. SAME MODEL, SAME PROMPTS, SAME ALGORITHM; ONLY THE LANGUAGE DIFFERS. SPEEDUP IS AGAINST EACH LANGUAGE'S OWN NON-SPECULATIVE BASELINE.

Language	Drafter	tok/s	tok/fwd	accept	speedup
Thai	`none`	34.89	1.009	—	1.000×
Thai	`lookup`	37.50	**2.002**	0.762	**1.075**×
Thai	`ngram`	32.06	1.384	0.465	0.919×
Thai	`hybrid`	36.36	**2.168**	0.741	**1.042**×
English	`none`	40.77	1.022	—	1.000×
English	`lookup`	38.14	1.337	0.531	0.935×
English	`ngram`	39.26	1.116	0.395	0.963×
English	`hybrid`	36.70	1.415	0.500	0.900×

an identical baseline moved 95.3 → 88.1 tok/s across back-to-back runs, and the sequential schedule produced an apparent 1.30× speedup for `lookup` on Thai summarization that *vanished to* 0.99× once configurations were interleaved at the prompt level. All results below are interleaved with three repeats.

Wide verification batches are not free. We initially assumed that on unified memory, verifying γ+1 tokens costs barely more than verifying one. Measured, a γ=2 step costs about 1.55× a single-token decode. Section VI replaces this single point with a curve.

C. Results

Draft acceptance is far higher in Thai than English at every task (summarization 0.942 against 0.583; grounded question answering 0.743 against 0.634; free-form chat 0.479 against 0.302; 1B, language-matched drafters). At 1B this does not convert into wall-clock gain—nothing beats 1.00×, because the fixed per-forward overhead eats the surplus. At 3B, where that overhead is amortised over more model work, it does.

Table IV is the paper's third result: **speculative decoding pays off for Thai and not for English**, on the same model, the same prompts and the same algorithm. The mechanism is Section III's token inflation operating in the language's favour—a copied span yields 2.00 tokens per verification pass in Thai against 1.34 in English, because Thai cuts identical content into roughly 2.2× more tokens, and because Section III-C's smaller pieces are individually easier to guess. The property that costs Thai throughput is partly recovered as drafting efficiency.

Two qualifications belong with the result. The gain is modest—7.5% at best—and appears only at 3B; its value lies less in the speedup than in showing that, with model and algorithm held fixed, the language decides its sign. And *the static Thai n-gram drafter we built is not the thing that works*: it loses everywhere on this backend (0.919×), and plain prompt lookup wins, because its tokens-per-forward clears the overhead threshold while the n-gram table's does not. Section VI shows that even this verdict is backend-specific. Outputs are not degenerate: the repeated-4-gram rate is 0.009–0.098 across Thai sets, so acceptance is not inflated by the model looping.

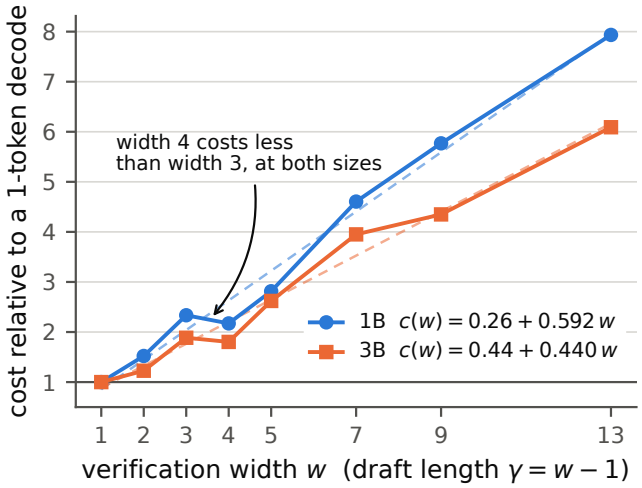


Fig. 3. Measured cost of a width-w verification forward, normalised to a single-token decode, with logits requested at every position; dashed lines are the least-squares fits. Widening is sub-linear in principle—the weights are streamed once either way—but the intercept is large enough that the first extra position costs over half a decode. The dip at width 3 inverts the ranking of draft lengths γ=2 and γ=3.

VI. A Cost Model, and What Replicates

The account above leans on a single measured constant—that a γ=2 step costs about 1.55× a plain decode. That is too weak a footing for a design rule, and it was measured on one machine. This section measures the full cost curve, restates profitability as an exact identity, and then re-runs the language comparison on a second backend with a model that has no Thai tuning at all.

A. What a wide verification forward costs

Let $c(w)$ be the wall-clock cost of a forward pass over w positions, normalised so $c(1) = 1$. We measure it directly: fixed batch widths, no drafter involved, 40 repetitions per width interleaved against each other at a KV depth of 512. Fig. 3 reports the result on an 8-thread x86-64 CPU for Llama-3.2 at two sizes.

Three things follow.

Profitability is an identity, not a rule of thumb. Measure time in units of a plain decode step and let $\bar{c}$ be the mean wall-clock cost of the speculative steps actually run, drafting included. Since the baseline emits essentially one token per forward (1.009 and 1.022 in Table IV), the speedup is

$$S = \frac{\text{tokens emitted} \,/\, \text{forward passes}}{\bar{c}}, \qquad (3)$$

and speculation pays exactly when tokens per forward exceed $\bar{c}$. The microbenchmark supplies the forward's share of $\bar{c}$: a drafter that fills its full width on every step pays $c(\gamma{+}1)$, one that often proposes fewer tokens, or none, runs narrower and cheaper forwards, and drafting and bookkeeping add on top.

TABLE V

CROSS-BACKEND REPLICATION ON SUMMARIZATION AT γ=2. THE M4 COLUMN IS TYPHOON2-3B ON METAL; THE CPU COLUMN IS STOCK LLAMA-3.2-3B, A MODEL WITH NO THAI TUNING, ON X86-64. UPPER BLOCK: SPEEDUP PER DRAFTER. LOWER BLOCK: `LOOKUP` STATISTICS, WITH MEAN STEP COST $\bar{c}$ = TOKENS PER FORWARD ÷ SPEEDUP (EQ. (3)). SPEEDUPS DIFFER IN MAGNITUDE; THE MECHANISM DOES NOT.

	Apple M4 (Metal)		x86-64 (CPU)	
	Thai	English	Thai	English
`lookup`	1.07×	0.94×	1.37×	1.08×
`hybrid`	1.04×	0.90×	1.44×	1.12×
`ngram`	0.92×	0.96×	1.17×	1.04×
tokens/forward	2.002	1.337	1.835	1.273
draft acceptance	0.762	0.531	0.727	0.446
step cost $\bar{c}$	1.86	1.43	1.34	1.18
Th ÷ En, tok/fwd	**1.50**		**1.44**	

$\bar{c}$ is therefore a property of drafter and workload as well as backend; Table V reports it as tokens per forward divided by speedup. Because c is close to affine, $c(w) \approx \alpha + \beta w$, the marginal cost of one extra draft position is β and the fixed per-step overhead is α; a drafter is worth widening only while its marginal acceptance exceeds β.

The curve flattens with model size, which is why the effect scales. The slope falls by a quarter from 1B to 3B ($0.592 \rightarrow 0.440$). A larger model spends proportionally more time streaming weights, which a wider batch reuses for free, so widening costs relatively less. This is the amortisation argument of Section V made quantitative, and it is consistent with the 1B-versus-3B pattern reported there, here on a different backend.

The curve is not monotonic. Width 3 is *more* expensive than width 4 at both model sizes (2.335 against 2.172; 1.886 against 1.803), despite being narrower. The load log points at the cause: tensors are repacked to an 8-wide blocked kernel that vectorises width 4 better than width 3. Since draft length γ uses width γ+1, this inverts the ranking of γ=2 against γ=3—and γ=2 is the default our own harness shipped with. We do not claim this dent exists on Metal; we claim that a default draft length chosen without measuring $c(w)$ on the target backend is chosen blind.

Two micro-measurements close off plausible alternatives. KV-cache trimming (2.0 μs) and the 128k-way argmax (239 μs) are negligible against a 23 ms decode. Requesting logits at every position rather than only the last costs 13.2% at 1B against 8.5% at 3B, scaling with the output head's share of the weights (21.3% and 12.3%) as it should.

B. Does the language effect replicate?

We re-ran the language comparison on the second backend using *stock Llama-3.2-3B-Instruct*, which shares Typhoon2's tokenizer but none of its Thai tuning. Configurations were interleaved within a pass and the whole comparison run in ABBA order (Thai, English, English, Thai), so each language is measured at both ends of the thermal envelope.

Table V separates what replicates from what does not.

The mechanism replicates. The Thai-to-English ratio of tokens-per-forward—the quantity the claim is actually about—is 1.50 on Metal and 1.44 on the CPU, across two backends, two vendors' silicon, and two models one of which was never tuned for Thai. Thai also shows the larger speedup with every drafter on both machines, except the M4's n-gram table (0.92× against 0.96×). The scaling claim replicates in direction: on the CPU, the best Thai summarization speedup rises from 1.23× at 1B to 1.44× at 3B, consistent with the flatter 3B curve of Fig. 3.

The magnitudes do not. Thai gains 1.37–1.44× on the CPU against 1.04–1.07× on the M4, and English is mildly *positive* on the CPU where it was negative on Metal. Identity (3) says where the difference lives. Tokens per forward are similar on the two machines (2.00 against 1.84 in Thai), so the gap is in the step cost: with `lookup`, $\bar{c}$ is 1.34 in Thai on the CPU against 1.86 on the M4. It is not a cheaper forward—at 3B the CPU's full-width $c(3)$ is 1.89, so its lookup steps must often run below full width—and because the two runs differ in model, device, offload, generation length and harness at once (Section VII), we do not attribute the gap further. Stated as "speculative decoding helps Thai more than English" the finding is robust; stated as "it helps Thai and hurts English" it is a statement about one backend.

The drafter verdict does not survive at all. On the CPU, `hybrid` overtakes `lookup` in both languages and `ngram` stops losing (1.17× in Thai). Section V's conclusion that the static n-gram drafter is not worth shipping is a single-machine conclusion, and we withdraw it as a general claim.

The ABBA schedule earned its keep: the Thai baseline fell 13.3% between slots (21.42 → 18.56 tok/s), enough drift to invent or destroy an effect of this size, yet the *speedups* barely moved (1.437 → 1.438 for `hybrid`). Dividing each drafter by the baseline of its own slot removes the drift almost exactly, and tokens-per-forward and acceptance come out bit-identical across slots, as greedy determinism requires.

C. Losslessness is a property of the backend

Every M4 run in this study reports zero mismatches against the greedy baseline, which is why Section V calls the emitted sequence the greedy one. The second backend shows that this holds for the M4 rather than in general. There, depending on task and draft length, up to 83% of generated sequences diverge from the greedy baseline (62.5% on Thai summarization at γ=2).

The cause is arithmetic, not logical. A width-1 and a width-w forward need not compute bit-identical logits, because changing the batch width changes the matrix-multiply kernel; where the top two logits are nearly tied, the argmax flips. Measured directly over 384 positions, disagreement occurs at 0.26% of positions at widths 5 and 9, and where it occurs the top-two margin is 0.295 against a typical 4.44. One flipped token makes everything downstream diverge, so over 96-token sequences a per-position rate of about 1% suffices for the 62.5% sequence-level rate; the 384-position probe, which

caught a single flip, cannot pin the rate down more tightly than that. The verification rule is unchanged; the backend's floating-point behaviour is not.

The correct statement is therefore: *the acceptance rule is lossless by construction, and whether an implementation realises that in bits depends on whether its kernels are width-invariant.* Anyone reporting lossless speculative decoding should verify it on the backend they ship, which costs one hash comparison per sequence.

VII. Scope and Limitations

Model scale and tokenizer family. Every experiment uses 1B- or 3B-parameter models sharing the Llama-3.2 tokenizer, and the calibration sweep uses a single 1B model. The Q2_K threshold is therefore a 1B result: larger models (8B–70B) may tolerate low bit-widths differently, and whether the point at which the importance matrix binds moves with scale is open. The speculative result rests on Thai fragmentation, so our account predicts a smaller advantage under tokenizers that fragment Thai less, such as Gemma-3 (TIF 1.60), and a larger one under Qwen-derived Thai models (2.57); neither prediction is tested here.

Languages. The calibration and speculative-decoding experiments are Thai-only. Table I shows Khmer and Burmese fragmenting far more than Thai, so we expect the mechanism to carry over to other *scriptio continua* languages, but that generalization is argued, not demonstrated; running one of them through Sections IV–VI is the natural next test.

The two backends are not a controlled A/B. They differ at once in model (Typhoon2-3B against stock Llama-3.2-3B), device and offload (Metal GPU on unified memory against an 8-thread CPU), generation length (160 against 96 new tokens), swept draft lengths and harness build. The design therefore supports within-machine comparisons—which configuration beats which on the same model and prompts—and the cross-machine agreement of ratios such as Thai-to-English tokens per forward. It does not support comparing absolute throughput or speedup magnitudes across machines, nor attributing their difference to any one factor, and we report the cross-backend result as a replication of a *mechanism* rather than of a number. The hardware is also narrow: one Apple M4 with 16 GB and one x86-64 CPU. Discrete GPUs and phone SoCs have different cost curves, and the configurations that clear their bar there may differ again.

An unresolved confound in the calibration reversal. Matching Thai and English calibration corpora on *token* count gave the Thai set 40% fewer documents (154 against 260), because Thai articles hit our per-article character cap sooner. Thai calibration therefore covers fewer distinct topics, and we cannot cleanly separate "language" from "topical diversity" in the Q3_K_M result; a re-run matched on document count as well would resolve it. This is itself a methodological warning: matched calibration corpora are hard to construct across languages of differing tokenization density, and matching on the obvious axis silently unmatches another.

Perplexity only, greedy only. Downstream benchmarks lack the headroom to corroborate the calibration results at 1B, so their practical significance rests on perplexity; the Thai calibration corpus is Wikipedia only. Decoding is greedy throughout, so "lossless" here is a statement about greedy decoding.

Thermal instability. Absolute throughput on the M4 drifted about 13% over a six-hour session, and the CPU machine was worse, losing 35% between two passes of one run. All speed comparisons are interleaved and therefore robust to this, but absolute tokens per second should not be compared across tables.

VIII. Conclusion

Within the setting we measured—1B and 3B models sharing the Llama-3.2 tokenizer, on an Apple M4 and an x86-64 CPU—Thai on-device inference inherits two English-shaped defaults. The throughput metric is straightforwardly misleading: an English user receives $2.7\times$ more text per second than a Thai user from the same runtime and model, $2.3\times$ from token count alone, and cross-language throughput claims should be reported in characters per second. The calibration default is more interesting than we expected—replacing English calibration with Thai is not generally beneficial, and at Q4_K_M and Q3_K_M it is a wash or slightly harmful, but at Q2_K, where Thai also becomes disproportionately fragile, it recovers 5.8% of Thai perplexity for free. The rule we can support is not "match the language" but "match the language once the bit budget is tight enough for the importance matrix to bind"; we located that point for one 1B model, and where it lies at larger scales is open.

The tokenization penalty, meanwhile, is partly self-correcting. Because Thai cuts content into more and individually more predictable tokens, model-free speculative decoding pays off more for Thai than for English, and that mechanism survives a change of backend, of silicon vendor and of model. What does not survive is the surrounding numerology: the cost of a speculative step, the best draft length, which drafter wins, and whether the implementation is lossless in bits are all properties of the backend, and each should be measured on the machine that will ship. The first three reduce to one identity—tokens per forward against the mean step cost—whose main ingredient, the cost curve $c(w)$, is cheap to measure; the fourth costs one hash comparison per sequence to check.

The effects are modest in absolute terms—5.75% perplexity at Q2_K, $1.07\times$ on the M4—and their value lies less in the gains than in correcting which defaults matter, and when.

Code and Data Availability

Code, prompts, benchmark task files, raw results, and the scripts that rebuild the calibration and drafter corpora and invert the perplexity trace are available at https://github.com/E27-25/thai-on-device.

Acknowledgment

The authors used Claude (Anthropic) for grammar checking and language correction of the manuscript. The authors reviewed all content and take full responsibility for it.

References

[1] G. Gerganov and `llama.cpp` contributors, "llama.cpp: LLM inference in C/C++," open-source software, 2023–2025. [Online]. Available: https://github.com/ggml-org/llama.cpp

[2] Ollama contributors, "Ollama," open-source software, 2023–2025. [Online]. Available: https://github.com/ollama/ollama

[3] E. Frantar *et al.*, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[4] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Machine Learning and Systems (MLSys)*, 2024.

[5] G. Xiao *et al.*, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[6] T. Dettmers *et al.*, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: k-bit inference scaling laws," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[8] P. Rust *et al.*, "How good is your tokenizer? On the monolingual performance of multilingual language models," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2021.

[9] A. Petrov *et al.*, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] O. Ahia *et al.*, "Do all languages cost the same? Tokenization in the era of commercial language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[11] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[12] C. Chen *et al.*, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[13] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[14] T. Cai *et al.*, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.

[15] Y. Fu *et al.*, "Break the sequential dependency of LLM inference using lookahead decoding," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.

[16] X. Miao *et al.*, "SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[17] A. Saxena, "Prompt lookup decoding," open-source software, 2023. [Online]. Available: https://github.com/apoorvumang/prompt-lookup-decoding

[18] L. Bandarkar *et al.*, "The Belebele benchmark: A parallel reading comprehension dataset in 122 language variants," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2024.

[19] NLLB Team *et al.*, "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.

[20] K. Pipatanakul *et al.*, "Typhoon 2: A family of open text and multimodal Thai large language models," arXiv:2412.13702, 2024.

[21] Llama Team, AI@Meta, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[22] A. Yang *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[23] Gemma Team, Google DeepMind, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[24] K. Pipatanakul *et al.*, "Typhoon: Thai large language models," arXiv:2312.13951, 2023.

[25] E. M. Ponti *et al.*, "XCOPA: A multilingual dataset for causal commonsense reasoning," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.